Phase 3: Core Ordering System Implementation

Olive Branch Bakes Ordering System Project

Phase 3 scenario and outcome

By the end of Phase 3, your Olive Branch Bakes application must take a real customer order from the welcome form to a saved, confirmed order that is still there after the app restarts.

Olive Branch Bakes scenario: The owner approved your Phase 2 structure and cottage bakery look. Now she wants to use the system for her holiday pre-order season. Customers will order cakes, pies, and seasonal treats for pickup. She needs the system to price orders correctly, stop orders she cannot bake in time, remember returning customers, and keep every order safely saved. The staff side of the system, including the administrator console, will be built by teams in Phase 4.

Phase outcome: Turn your Phase 2 stubs and prototypes into a working Windows ordering system with menu browsing, a cart, pricing, pickup scheduling, customer accounts, and saved orders that survive restarting the app. Your grade comes mainly from explaining that code and justifying why you built it the way you did: 75% of Phase 3 points are for explanation.

Phase 3 builds on YOUR Phase 2. You must keep the folder structure, class names, form names, and navigation diagram you submitted in Phase 2. If you change any of them, record the change and the reason in your Change Log (see Required work). Work that does not trace back to your own Phase 2 submission will not be accepted.

Length: 4 weeks, with four checkpoints (3A to 3D).

Phase 3 focus: explain and justify your code

Phase 3 grades your understanding, not just a program that runs. 75 of the 100 points come from explaining what your code does and justifying why you chose to write it that way; only 25 points come from the working system and process evidence.

You will still build a complete ordering system. But a working program you cannot explain earns at most 25 points, while a partly working program you can explain and defend can still earn a strong grade.

How you show understanding	What it means for you	Points
Live code defense	A 15-minute, one-on-one session where you explain, trace, predict, and change your own code.	25
Design Decision Record	A written record of at least 10 code decisions: what the code does, why you chose it, and what you rejected.	20
Weekly explanation videos and checkpoints	Each week you explain the code you added and answer "why" questions in class.	20
Final video explanation	A walkthrough of your Variant Card logic and architecture, justified line by line.	10

AI tool policy: You may use AI tools to help you write code, find bugs, or learn C# and WinForms. You are fully responsible for every line you submit. Mark any section that came mostly from an AI tool in your Design Decision Record, and explain it as if you wrote it yourself. Code you cannot explain earns no credit, wherever it came from. Your Variant Card rules come from your student ID, so a generic AI solution will not match your test table.

Explaining your code

Every explanation you give, written or spoken, is scored on the same four-level scale. Aim for Level 4 on every decision.

Level	What you can do	Credit
4: Justify	Explain what the code does, how it works line by line, why you chose this approach, one alternative and why you rejected it, and what would happen if a value or line changed	100%
3: Explain	Explain what the code does and how it works line by line, but not why it was chosen over another approach	75%
2: Describe	Say what the code does in general terms, without explaining how it works	40%
1: Cannot explain	Read the code aloud or cannot say what it does	0%

Design Decision Record (docs/DesignDecisions.md)

Write at least 10 decision entries, added as you build (at least 3 new entries each week). Each entry has six parts:

1. **Decision:** one sentence naming the choice, such as "Store money as decimal."
2. **Where:** the file and method, with a GitHub link to the commit.
3. **What it does:** in your own words, with no copied code comments.
4. **Why:** the reason this choice fits Olive Branch Bakes and your Variant Card.
5. **Alternative rejected:** one other way you could have done it, and why you did not.
6. **Source and test:** whether you wrote it, adapted it, or used an AI tool, and which test row proves it works.

Required decisions (8): decimal for money; where and how your Pricing rule is applied; how your Operations rule is checked; your data storage format (JSON or SQLite); password hashing; confirmation number generation; how forms get data from services; how input validation is organized. **Your own decisions (at least 2):** any other choice you made, such as a control type for touch use or how the cart stores lines.

Your Bakery Variant Card

The Bakery Variant Card gives each student a different business rules.

How it works: each student looks at the last three digits of their student ID. Each digit picks one rule from the table:

Digit	Picks	Example choices
Third-to-last	A seasonal menu item	Pecan pie, gingerbread loaf, king cake…
Second-to-last	An operations rule (when or how much a customer can order)	Max 12 pies per day, closed Mondays, 72 hours' notice for cakes…
Last	A pricing rule (a discount or fee)	Baker's dozen, 10% loyalty discount, rush fee…

Example: a student with ID 4071**852** gets:

- **8** → Maple pecan sticky bun as their seasonal item
- **5** → Pickup slots: closed Mondays, 30-minute slots, max 4 orders per slot
- **2** → Loyalty pricing: 10% off for customers with 3+ past orders

The student with ID ending in 317, would build a sweet potato pie, a lead-time rule, and a "13th cookie free" discount instead.

Your last three student ID digits set three rules that only you know. Show your Variant Card on your screen at the start of your video.

Example: student ID 4071**852**

→ third-to-last digit **8**, second-to-last digit **5**, last digit **2**

→ **Rules**: Seasonal item 8, Operations rule O3, Pricing rule P2.
// Maple pecan sticky bun (each), O3 Pickup slots, P2 Loyalty

The Variant Card shows up in every stage of Phase 3. Each student builds their own three rules into the ordering system and then must prove and explain them.

When	What You Do with Your Variant Card
Day 1	Works out the three rules from your ID digits and write them in docs/VariantCard.md in their GitHub repository.
Week 1 (3A)	Writes test rows by hand that target your own rules: just below the discount threshold, exactly at it, one past the operations limit, and so on. Calculate the expected totals before writing any pricing code.
Week 2 (3B)	Add your seasonal item to the menu and shows it disappearing outside its season dates.
Week 3 (3C)	Builds your pricing rule into PricingService and your operations rule into SchedulingService. The weekly video runs the test rows live against your hand calculations.
Design Decision Record	Two of the required entries cover where the pricing rule lives and how the operations rule is checked, and why you built it that way.
Week 4 final video	Starts by stating the three rules. Explains the pricing and scheduling code line by line and run test rows T1, T3, T7, and T10.
Code defense	You demonstrate your three rules. For example: “Show where the 4-orders-per-slot limit is checked.” “What happens if I change 48 hours to 24?” “Predict this cart’s total under your loyalty rule.”

Seasonal signature item (third-to-last digit)

Add this item to your menu. It must be orderable only between its season dates, which are stored in your menu data file, not hard-coded in a form.

Digit	Item	Price	Season
0	Apple cider donut (each)	$1.75	Sep 15 to Nov 30
1	Pumpkin swirl bread (loaf)	$8.50	Oct 1 to Nov 30
2	Pecan pie (9 inch)	$24.00	Nov 1 to Dec 31

Digit	Item	Price	Season
3	Sweet potato pie (9 inch)	$21.00	Nov 1 to Dec 31
4	Gingerbread loaf	$9.25	Nov 15 to Jan 5
5	Cranberry orange scone (each)	$3.25	Nov 1 to Jan 15
6	Peppermint brownie (each)	$3.50	Dec 1 to Dec 31
7	King cake (serves 12)	$32.00	Jan 6 to Mar 4
8	Maple pecan sticky bun (each)	$4.25	Oct 15 to Dec 31
9	Honey lavender shortbread (6 pack)	$11.00	Oct 1 to Dec 31

Operations rule (second-to-last digit)

Digits	Rule	What the system must do
0 or 1	**O1 Pie capacity**	No more than 12 pies total across all orders for one pickup date. Block the order and show how many pies are still available for that date.
2 or 3	**O2 Lead times**	Sourdough and all yeast breads need 48 hours' notice; everything else needs 24 hours. Show the earliest valid pickup time for the cart.
4 or 5	**O3 Pickup slots**	Closed Mondays. Sunday pickups 7:00 a.m. to 1:00 p.m.; other days 7:00 a.m. to 5:30 p.m. Pickups use 30-minute slots, with at most 4 orders per slot. Full slots must not be selectable.
6 or 7	**O4 Custom cakes**	Custom decorated cakes need 72 hours' notice and a message of 40 characters or fewer. The order records a 25% deposit due now and a balance due at pickup.
8 or 9	**O5 Quantity limits**	Guests may order at most 6 of any one item. Signed-in customers may order up to 24. The limit message must tell guests that signing in raises the limit.

Pricing rule (last digit)

Digits	Rule	What the system must do
0 or 1	**P1 Baker's dozen**	For each cookie or muffin item, every 13th unit is free (13 for the price of 12, 26 for the price of 24).
2 or 3	**P2 Loyalty**	Signed-in customers with 3 or more previous orders in their order history get 10% off the subtotal. Guests never qualify.
4 or 5	**P3 Bread and butter pair**	Each loaf of bread paired with one jar of honey butter takes $2.00 off. Unpaired items get no discount.
6 or 7	**P4 Volume tiers**	Subtotal $50.00 to $99.99 gets 5% off. Subtotal $100.00 or more gets 12% off.
8 or 9	**P5 Plan ahead**	Orders placed 7 or more days before pickup get $0.50 off each pie and cake. Orders placed less than 48 hours before pickup add a $5.00 rush fee.

Discounts apply before tax. Fees are taxable. Round money to the nearest cent only when you display or save a total, never in the middle of a calculation.

Required work

Not in Phase 3: the administrator login and administrator console. Your Phase 4 team builds them, using the order statuses, data files, and settings file you create now.

Models (complete your Phase 2 stubs)

- ☐ Implement the properties, constructors, and validation for your domain classes. At minimum: menu item, category, customer, order, order line, and pickup time (or the names you used in Phase 2).
- ☐ Order lines calculate their own line total; orders calculate subtotal, discount, fees, tax, and total.
- ☐ Use decimal for all money. Never use double or float for prices.

Services (the business logic lives here, not in the forms)

- ☐ **MenuService:** loads the menu, filters by category, and hides inactive and out-of-season items.

- ☐ **PricingService:** applies your Pricing rule, then tax. It must not reference any form or control.
- ☐ **SchedulingService:** validates pickup dates and times and applies your Operations rule.
- ☐ **AccountService:** creates accounts, signs customers in and out, and stores passwords as salted hashes, never as plain text.
- ☐ **OrderService:** places orders, assigns confirmation numbers in the format OBB-YYMMDD-###, looks up order history, and saves every new order with the status Received.
- ☐ **DataStore:** saves and loads menu items, customers, orders, and settings to JSON files (or a local SQLite database). Data must survive closing and reopening the app. Store the tax rate (8.5%) and your Operations rule values in a settings file, not in code.

Customer forms (from your Phase 2 prototypes)

- ☐ **Welcome form:** order as guest, sign in, or create account. Keep the Administrator button from your Phase 2 navigation, but have it show a "Coming in Phase 4" message.
- ☐ **Customer ordering form:** browse by category, see price and description, choose quantity with large touch-friendly buttons, add to cart, change or remove cart lines, and see a running subtotal.
- ☐ **Account form:** create an account (name, email, phone, password) with validation; sign in; view past orders.
- ☐ **Order summary form:** list lines, discount, fees, tax, and total; choose a pickup date and time; enter notes; place the order; show the confirmation number.

Quality

- ☐ Every input is validated with a friendly message in plain language. The app never crashes on bad input, an empty cart, or a missing data file.
- ☐ Keep your Phase 2 palette, readable type (at least 12 pt for body text), logical tab order.
- ☐ Seed data: at least 15 menu items across at least 4 categories, plus your seasonal item, 3 customer accounts, and 8 sample orders across at least 3 pickup dates.
- ☐ **Change Log:** a table listing every change from your Phase 2 design (what changed, why, and the date).
- ☐ **Developer Journal:** one entry per work session (see GitHub setup and Final submission).

Core business rules for all students

These rules apply to everyone, on top of your Variant Card. When a rule here conflicts with your Variant Card, your Variant Card wins.

Price calculation order. Your PricingService must follow this order exactly:

1. Line total = unit price × quantity, for each line.
2. Subtotal = sum of line totals.
3. Apply your Pricing rule discount (if any) to get the discounted subtotal.
4. Add fees (rush fee, if your rule has one).
5. Tax = (discounted subtotal + fees) × tax rate.
6. Total = discounted subtotal + fees + tax, rounded to the nearest cent.

Cart rules.

- Quantity must be a whole number from 1 to the limit for that item. Adding an item already in the cart increases its quantity instead of adding a second line.
- An order cannot be placed with an empty cart, or without a valid pickup date and time.
- Prices are copied into the order line when the order is placed. Changing a menu price later must not change past orders.

Pickup rules.

- Pickup must be at least 24 hours after the order is placed (unless your Operations rule sets a longer time) and no more than 30 days ahead.
- The bakery is closed on Thanksgiving Day and December 25. These dates must be blocked.

Order status rules. Every new order is saved with the status Received. Define the full set of statuses in your Order model now (Received, Baking, Ready, Picked Up, Cancelled), because your Phase 4 team will build the screens that change them. In Phase 3, customers see the status in their order history but cannot change it.

Account rules.

- Email addresses are unique and are not case-sensitive.
- Passwords are at least 8 characters with at least one number.
- A signed-in customer's name and phone fill in automatically on the order summary.

Design rule. No business rule may live only inside a form's event handler. Forms collect input and display results; services make decisions. During your code defense you will be asked to show where each rule lives.

GitHub setup and workflow

You will track every code change in a private GitHub repository. Your instructor reviews it each week, and your Phase 4 team will build on these repositories, so keep yours clean and complete.

One-time setup (finish before Checkpoint 3A)

1. **Create a GitHub account** at github.com with your school email. Choose a professional username, such as your first initial and last name.
2. **Sign in from Visual Studio.** Choose File > Account Settings, then add your GitHub account under All Accounts.
3. **Open your Phase 2 solution** in Visual Studio and choose Git > Create Git Repository.
4. **Create the repository.** Under Push to a new remote, choose GitHub and your account. Name it obb-ordering-lastname-firstname, check Private repository, and keep the Visual Studio .gitignore template. Choose Create and Push.
5. **Add your instructor.** On github.com, open your repository, go to Settings > Collaborators > Add people, and add the instructor username posted in the course site.
6. **Add a docs folder** inside your solution folder with these files: VariantCard.md, DeveloperJournal.md, ChangeLog.md, and your test table (TestTable.xlsx or TestTable.md). Commit and push them.
7. **Check it worked.** Refresh your repository page. Your solution files and docs folder should appear; bin, obj, and .vs folders should not.

Everyday workflow (every work session)

1. Make one small change that works, such as one method, one validation, or one control.
2. Build and run to confirm nothing broke.
3. Open View > Git Changes and review the files that changed.
4. Write a commit message as a verb, what changed, and where: "Add pie capacity check to SchedulingService" or "Fix cart total not updating after remove".
5. Choose Commit All, then Push (or Commit All and Push from the drop-down).
6. Add your Developer Journal entry for the session, then commit and push it too.

Rules for your repository:

- All code changes must come from commits in Visual Studio. Do not upload code by dragging files into the GitHub website; it erases the history your grade depends on.
- Never delete and recreate your repository, and never force push. If something goes wrong, ask your instructor first.

- Commit messages like "update", "stuff", or "final" do not count toward your commit minimum.
- If a push is rejected, choose Git > Pull, then push again.

Weekly release (end of every week)

A release is a labeled snapshot of your code that marks exactly what you submitted that week.

1. Confirm everything is committed and pushed.
2. On your repository page on github.com, choose Releases, then Draft a new release.
3. Under Choose a tag, type the tag name for the week (week1-3A, week2-3B, week3-3C, or phase3-final) and choose to create it on publish.
4. Title it, for example "Week 1: Checkpoint 3A". In the description, write 2 to 3 sentences on what works and what does not yet.
5. Publish the release. Copy its web address and download the Source code (zip) file GitHub attaches to it. You submit both each week.

Weekly submissions

Every week you submit three things to the course site by the weekly due date: your GitHub release link with its Source code ZIP, a narrated progress video, and the commits that produced that week's work. GitHub timestamps show exactly when each commit and release was made; a release published after the due date is late.

Week	Code (GitHub release)	Progress video	Commits
1: Checkpoint 3A	Release week1-3A: completed model classes, DataStore saving and reloading seed data, docs folder with Variant Card and hand-calculated test table	3 to 5 min: show your Variant Card; save, close, reopen, and show the data reloaded; show your hand calculations; scroll your GitHub commit list for the week	At least 5 meaningful commits on at least 2 different days
2: Checkpoint 3B	Release week2-3B: menu browsing by category, seasonal item visibility, cart add, change, and remove, running subtotal	3 to 5 min: run the ordering form; show your seasonal item hidden outside its season (change the date to test); open one method you wrote this week and explain it line by line	At least 5 meaningful commits on at least 2 different days

Week	Code (GitHub release)	Progress video	Commits
3: Checkpoint 3C	Release week3-3C: Pricing and Operations rules, order summary, accounts, saved orders with confirmation numbers	5 to 7 min: run test rows T1 to T7 live next to your test table; create an account and sign in; place an order and show its confirmation number; explain your Variant Card logic in the service where it lives	At least 5 meaningful commits on at least 2 different days
4: Checkpoint 3D and final	Release phase3-final: complete system, final test table, Developer Journal, and Change Log in docs	Final video, 12 to 15 min (see Final submission)	At least 5 meaningful commits on at least 2 different days

Every progress video must:

- Start with you stating your name and the date, with your GitHub repository page on screen.
- Be narrated in your own voice throughout; no music-only or silent recordings.
- Show the version of the app that matches that week's release, not a later one. Spend at least two-thirds of the video explaining code: open the methods you added, explain how they work, and justify why you chose that approach, using the four-level scale.

What to upload each week: the release web address, the Source code ZIP downloaded from that release, and your video file or link. Your instructor also checks your Developer Journal entries for the week in your repository.

Checkpoints

Each checkpoint is a 3 to 5 minute demo at your own computer. Most checkpoint time is spent on explanation: your instructor will point to code you added that week and ask what it does and why you chose it.

Checkpoint	Due	Show working	Example live change request
3A Models, data, and test table	End of week 1	Completed model classes; DataStore saves and reloads seed data; your hand-calculated test table (next section), done before pricing code exists.	"Add an Allergens property to menu items and make it save and load."
3B Customer ordering flow	End of week 2	Menu browsing by category, seasonal item visibility, cart add, change, and remove, running subtotal.	"Make the cart show the item count on the Checkout button."
3C Pricing, scheduling, accounts	End of week 3	Your Pricing and Operations rules working; order summary with correct totals matching your test table; account create and sign in; order saved with confirmation number.	"Change your discount threshold and show me which test rows now change."
3D Saved data, history, and polish	End of week 4	Orders, accounts, and settings survive closing and reopening; signed-in customers see their order history and status; every invalid input gives a friendly message; touch use and tab order checked on a tablet or touchscreen.	"Add a filter so order history shows only upcoming pickups."

What the instructor checks at every checkpoint:

- Your GitHub commits and release since the last checkpoint (at least 5 new meaningful commits).
- Your Developer Journal entries since the last checkpoint.
- That you can point to the file and method where the work you just showed lives.

Test table and acceptance tests

You will prove your system works by matching results you calculated by hand before you wrote the code. Submit a photo or scan of your handwritten work, or a spreadsheet with the formulas visible, at Checkpoint 3A.

Baseline menu items

Your menu must include these 7 items at these prices, so everyone shares a common starting point. Add your own items and your seasonal item to reach at least 16.

Item	Category	Price
Sourdough loaf	Breads	$7.50
Chocolate chip cookie	Cookies	$2.25
Blueberry muffin	Muffins	$3.00
Cinnamon roll	Pastries	$3.75
Apple pie (9 inch)	Pies	$22.00
Custom decorated cake	Cakes	$45.00
Honey butter (jar)	Extras	$5.00

Required test rows

Write at least 12 test rows. Rows T1 to T10 are required; design the inputs for T2 to T10 yourself to fit your Variant Card. Each row lists: test ID, what it checks, exact inputs (items, quantities, signed in or guest, order date and time, pickup date and time), expected result, actual result, and pass or fail.

ID	What the row must test
T1	Baseline: 2 sourdough loaves, 6 chocolate chip cookies, 1 apple pie, guest, pickup 3 days out
T2	Just below the point where your Pricing rule starts to apply
T3	Exactly at the point where your Pricing rule starts to apply
T4	Well past that point, or the rule applied more than once
T5	Same cart as guest and as signed-in customer
T6	Your Operations rule at its limit, allowed
T7	Your Operations rule one step past its limit, blocked with the correct message
T8	Your seasonal item one day outside its season
T9	Empty cart, or quantity of 0 or a letter
T10	Pickup on December 25

Worked example of T1 for a student whose rule does not apply to this cart: subtotal $15.00 + $13.50 + $22.00 = $50.50; discount $0.00; fees $0.00; tax $50.50 × 0.085 = $4.2925; total $54.7925, displayed as **$54.79**. Your T1 may differ if your Pricing rule applies to this cart. If it does, show why.

Phase 3 acceptance tests

- ☐ A guest can browse, build a cart, choose a valid pickup time, and place an order that receives a confirmation number.
- ☐ Every row of your test table produces your hand-calculated result in the running app.
- ☐ Your Pricing rule and Operations rule behave exactly as your Variant Card describes, including at their boundaries.
- ☐ Orders, accounts, and settings are still there after closing and reopening the app.
- ☐ A returning customer can sign in, see past orders, and have their details filled in.
- ☐ Your GitHub repository shows a release and at least 5 meaningful commits for each of the 4 weeks.
- ☐ Invalid input of any kind produces a friendly message and never crashes the app.
- ☐ The design still follows your Phase 2 bakery theme and works by touch on a Windows tablet or touchscreen.

Final submission (end of week 4)

Your final submission is your week 4 weekly submission plus the documents below, all inside your GitHub repository's docs folder.

1. GitHub release phase3-final

Submit the release web address and its Source code ZIP. Your instructor must already be a collaborator on the repository. Across Phase 3 your history must show at least 20 meaningful commits and four weekly releases.

2. Test table (docs/TestTable)

Every row completed with expected and actual results, plus the photo, scan, or spreadsheet of your hand calculations.

3. Developer Journal (docs/DeveloperJournal.md)

One entry per work session, 3 to 6 sentences each, with the date. Each entry answers:

- What did I build or change, and which commits hold it?

- What went wrong, and how did I figure it out?
- If I used an AI tool, what did I ask and what did I learn from it?

4. Change Log (docs/ChangeLog.md)

A table of every change from your Phase 2 design: date, what changed (class, form, folder, or navigation), and why.

5. Design Decision Record (docs/DesignDecisions.md)

At least 10 complete entries, written as described in Explaining your code, with at least 3 dated in each week.

6. Final video demonstration (12 to 15 minutes)

Your voice must narrate throughout. Your webcam must be on in a corner of the screen for at least the code walkthrough. The video must:

1. Show your Variant Card and state your three rules in your own words.
2. Run a complete guest order from the welcome form to the confirmation number.
3. Run test rows T1, T3, T7, and T10 live and compare each result with your hand-calculated table on screen.
4. Create an account, place an order while signed in, then close and reopen the app to prove the order, its status, and the account were saved.
5. Open your GitHub repository and walk through your commit history and four releases, pointing out one commit from each week and what it changed.
6. Open PricingService and SchedulingService and explain your Variant Card logic line by line.
7. Complete the **Video Change of the Week**, a small change your instructor posts 48 hours before the due date. Make the change on camera, run it, commit it, and push it.
8. Explain one bug you found during Phase 3 and how you fixed it, using your commit history or journal as evidence.
9. Explain how the finished system supports Windows computer and tablet use.

Code defense and grading

Phase 3 ends with a 15-minute, one-on-one code defense, worth 25 points, on your own submitted project. You may use your project and notes, but no AI tools or outside help. Your instructor will choose three of these tasks and score each on the four-level scale:

- **Explain:** walk through a method the instructor points to and say what each part does.
- **Trace:** follow a value from a button tap on the ordering form to the line where it is saved to the data file.

- **Predict:** calculate by hand the total for a new cart the instructor gives you, then run it to check.
- **Change:** make a small change, such as adding a category or adjusting a rule value, and run it.
- **Debug:** your instructor will plant one bug in a copy of your project. Find and fix it.

Grading rubric (100 points)

Category	Area	Points	Full credit looks like
Explanation	Live code defense	25	All three defense tasks at Level 4: you explain how your code works and justify why you chose it over an alternative
Explanation	Design Decision Record	20	10+ complete entries, all six parts, written in your own words, added weekly, with honest source notes
Explanation	Weekly explanation videos and checkpoints	20	Each week (5 points): video spends two-thirds of its time explaining code at Level 3 or 4, and you answer the instructor's "why" questions at checkpoint
Explanation	Final video explanation	10	Variant Card logic and architecture explained line by line and justified; all 9 required video parts shown
Implementation	Working ordering system and test table	15	Customer, account, and saving features work; every test row matches your hand calculation
Implementation	GitHub process evidence	10	20+ meaningful commits (5+ a week on 2+ days), four weekly releases, Developer Journal and Change Log complete

Explanation is required for implementation credit: if you score Level 1 on any feature during the code defense, that feature earns no implementation points. If your rules do not match your own Variant Card, the test table scores zero. Your instructor may hold a second defense before applying either rule.